

SOLARIS: A Solar Optimized Land Autonomous Recharge and Intelligence System

Nathan Hammond, Rafael Puig, Garret
Fortier, Ryan Markowitz

Dept. of Electrical Engineering and
Computer Science, University of Central
Florida, Orlando, Florida, 32816-2450

Abstract — This paper presents the design of SOLARIS, an autonomous ground robot built to make solar energy collection mobile instead of stationary. A fixed solar panel loses power whenever the sun moves or shade rolls in, and it has no way to do anything about it. SOLARIS treats that as a problem worth solving by driving itself toward better light and steering a two axis solar panel to track the sun. The decision to relocate is made greedily from onboard power readings and weather data rather than a full optimization. When the panel's output drops and the sky is clear, the robot tries moving, checks whether the move actually recovered power, and gives up after a few tries if it does not. The system is built around an ESP32-S3 microcontroller that runs the firmware, reads a suite of light, motion, and distance sensors, drives four wheel motors and a pan and tilt panel assembly, and reports its state to a phone over Bluetooth Low Energy. A companion mobile application handles user authentication, live telemetry, and manual driving, while a cloud backend stores logged data and feeds weather forecasts back down to the robot. This paper covers the requirements, the hardware, the software architecture and its state logic, the wireless protocol, the application and backend stack, and the standards and constraints that shaped the design. It closes with the subsystem testing done so far and the work that remains.

Index Terms — *Autonomous robot, solar tracking, embedded systems, ESP32, Bluetooth Low Energy, MPPT, power aware design.*

I. INTRODUCTION

Solar power keeps getting cheaper and batteries keep getting better, but the panels themselves still mostly sit in one place. A stationary panel is only pointed the right way for part of the day, and if something shades it, the output drops and stays down until the sun moves on its own. SOLARIS, which stands for Solar Optimized Land Autonomous Recharge and Intelligence System, is our attempt to give a solar panel the ability to react. It is a self

contained robot that carries its own panel, its own battery, and its own sensors, and it decides on its own when to move, where to move, and how to angle the panel.

The idea that separates SOLARIS from a normal solar tracker is that moving has to be worth it. Driving the robot costs energy and rotating the panel costs energy, so there is no point chasing a brighter patch of ground if getting there burns more than the panel will make once it arrives. We do not try to solve that perfectly. The decision to move is greedy: the robot watches its own power output and its weather forecast, and when the numbers say moving is probably worth a shot, it tries. It is not guaranteed to be right every time, but a few cheap safeguards keep it from wasting much energy on a bad guess. That power aware behavior is the thread that runs through the whole design, from the motors we picked to the logic in the firmware.

There is no product on the market that does exactly this. The closest thing is a concept from Jackery called the Solar Mars Bot, an autonomous solar robot that is supposed to drive toward stronger sunlight and fold out large panels to charge an onboard battery [1]. It has not shipped, and its control and navigation details are not public, but the fact that a company is working on the idea tells us the design space is worth being in. Below the commercial level there are hobby and STEM kits that tilt a small panel with servo motors based on light readings, but none of them actually drive anywhere. SOLARIS pushes past the tilt only approach by making the whole platform mobile and by making the decision to move a power aware one.

Beyond the robot itself, this project is a full stack learning exercise for a mixed team of Electrical and Computer Engineering students. It forces hardware and software to meet in the middle: low power electronics on one side, embedded firmware and a networked mobile application on the other. The rest of this paper walks through how the two sides come together. Section II lays out the requirements. Section III covers the hardware. Section IV covers the software architecture and the decision logic that runs the robot. Section V covers the wireless link. Section VI covers the mobile application and backend. Section VII covers the standards and design constraints. Section VIII presents testing results, and Section IX concludes.

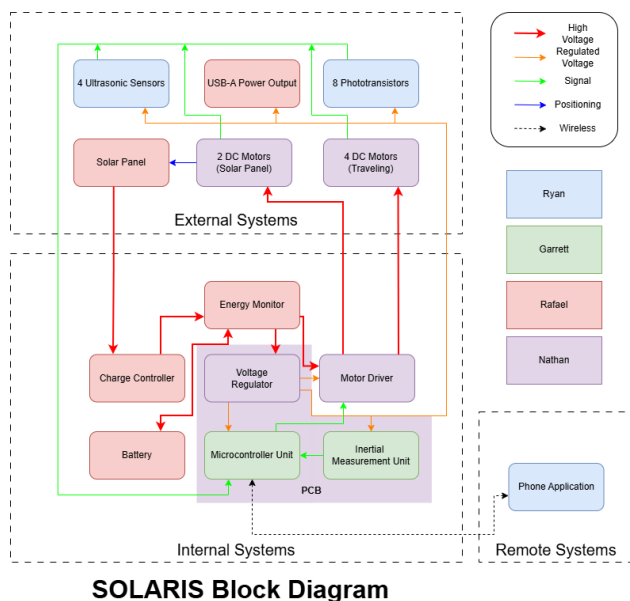


Fig. 1 SOLARIS system level block diagram

II. REQUIREMENTS AND SPECIFICATIONS

We split the project into basic and advanced goals so the team always had a working baseline to fall back on. The basic goals are the ones the robot has to hit to be called done: harvest solar energy into an onboard battery, provide that energy back to a user through a USB port, and run for a stretch of time with no wall power and no person touching it. The advanced goals add the parts that make it interesting, which are the autonomous driving, the multiple operating modes, the wireless control and telemetry, and the weather feed that informs the move decision. Everything described in this paper falls into one of those two tiers.

The energy targets set the tone for the rest of the design. After a good sunny day the system is meant to generate at least 15 watt hours of usable energy pulled from the battery through a USB-A port. On the mobility side the robot tops out around 0.3 m/s, cruises around 0.1 m/s, and is kept under 50 lbs. The solar panel gets a full 360 degrees of pan and 30 degrees of tilt so it can point at the sun through the day without the whole robot having to reorient.

Specification	Description	Target Values
Energy For User	After a sunny, optimal day, the system shall provide the end user with at least 15 watt-hours (Wh) of usable energy from the battery.	≥ 15 Wh/day
Max Robot Speed	Normal forward robot speed	0.3 m/s
Ultrasonic Sensor Detection	Sensor detection range	< 5 m
Solar Panel Tilt	Tilt axis range of motion	30 degrees
Solar Panel Rotation	Pan axis range of motion	360 degrees
BLE Range	Range of BLE to user phone	< 5 m

Fig. 2 SOLARIS overall system specifications table

At the component level, the specs come from the parts we ended up using. Every moving part on the robot is driven by a goBILDA Yellow Jacket brushed DC planetary gear motor, which keeps the drive electronics and the mounting hardware consistent across the whole platform. The four drive motors are the goBILDA 5204 series, rated at 12 V and 18.14 Nm at 60 RPM through a 139:1 planetary gearbox, paired with 4 inch wheels to hit the target cruising speed with plenty of torque margin for grass and startup inertia. The panel pan and tilt motors are the goBILDA 5203 series, rated at 12 V and 24.5 Nm at 30 RPM through a 188:1 planetary gearbox, since holding and swinging the panel against imbalance and wind needs more torque and less speed than rolling a wheel. Both motors run on 12 V, which is the battery's output level, so no extra conditioning is needed to drive them, and both ship with integrated quadrature encoders that run on 3.3 V logic and tie straight into the ESP32-S3 for closed loop speed control and odometry. A 20 W, 12 V solar panel is the charging source, feeding a battery sized at 256 Wh. The motors run off H-bridge driver boards, and the logic is fed by two switching regulators that produce 3.3 V and 5 V rails, with the 5 V rail feeding the ultrasonic sensors, energy monitor, and motor drivers and the 3.3 V rail feeding the logic, light sensors and IMU. Bluetooth Low Energy handles the link to the phone at a working range of a few meters.

III. HARDWARE DESIGN

The hardware is built as a set of modules that all hang off the ESP32-S3. We chose the ESP32-S3 because it gives us Bluetooth Low Energy, plenty of GPIO, hardware peripherals for motor control and encoder counting, and a large and well documented driver ecosystem that keeps firmware development moving instead of stalling on missing support.

A. Power System

Power is the whole reason the project exists, so the power path got the most attention. The solar panel feeds a Genasun charge controller that uses Maximum Power Point Tracking. Unlike a plain PWM controller, the MPPT controller constantly adjusts the electrical operating point of the panel to pull the most power available for the current sunlight and temperature. That difference is what lets the battery gain outrun the robot's baseline draw over a day.

From the battery, the design steps the roughly 12.8 V down to regulated 3.3 V and 5 V rails using LM2679 based regulators, with the 5 V rail feeding the ultrasonic sensors and the 3.3 V rail feeding the logic and light sensors. Because the drive motors are the noisiest device on the board electrically, the motor power bus is kept separate from the sensor and logic supplies so that current spikes and back EMF from the motors do not corrupt I2C sensor traffic or the radio.

The whole power aware strategy rests on being able to measure net power in real time. The INA228 monitor reports bus voltage and current based on a known shunt resistance value. We are able to extrapolate this data into instantaneous net power

$$P(t) = V(t) \cdot I(t) \quad (1)$$

where a positive value means the robot is drawing more energy than the panel can supply and a negative value means the panel is powering the system and putting energy back into the battery. Integrating that over a window gives the energy gained or spent,

$$E = \int P(t) dt \approx \sum P[k] \cdot \Delta t \quad (2)$$

which the firmware approximates as a running sum of samples. The design target is to stay energy positive across a day, meaning the harvested energy has to cover everything the robot spends on moving, sensing, and communicating and still leave the user their share,

$$E_{\text{gen}} - E_{\text{used}} \geq E_{\text{user}} \quad (3)$$

Equation (3) is the rule every design decision gets checked against. Energy generated allows solaris to continue functioning in between periods of darkness or make up for energy spent moving.

B. Sensing

SOLARIS reads its environment three ways: light, motion, and distance, plus a dedicated channel for its own power. To keep from burning a GPIO pin on every sensor, the two large sensor banks are read through multiplexers. The eight light sensors sit behind an 8:1 mux and the four ultrasonic sensors sit behind a 4:1 mux, and the firmware enumerates through each mux channel via GPIO to collect a full set of readings off a single line.

Light is what the robot moves toward, so the light subsystem does a lot of the work. It uses eight Vishay TEPT5600 silicon NPN phototransistors, which respond to visible light from 440 to 800 nm with a peak near 570 nm, close to the human eye. Each one is wired with its collector on the 3.3 V rail and its emitter to ground through a load resistor. When light hits the phototransistor it passes a collector current proportional to the illuminance, and that current across the load resistor develops a voltage that rises with light intensity. That voltage is routed through the 8:1 mux, and the firmware steps through the mux channels and samples each one on an ADC input of the ESP32-S3, so it reads all eight sensors in real time off one line. Spreading eight of these around the robot lets it compare readings from different directions and figure out where the strongest light is. We picked the TEPT5600 partly for its narrow acceptance angle, about plus or minus 20 degrees at half sensitivity, which rejects off axis light and makes each sensor report on the light more or less in front of it. The load resistor value is not finalized yet. Our early schematic used 10 k Ω as a placeholder, but in practice it will likely end up considerably lower to bring the gain down and keep the sensors from saturating under direct sunlight, and the final value will come out of bench testing. The 3.3 V rail keeps the parts inside their 6 V collector to emitter limit, and the resistor will be chosen to keep the collector current well under the 20 mA maximum for any sunlight the robot will see outdoors.

Motion and heading come from an ICM-20948 inertial measurement unit, which the firmware reads directly over I2C. It gives raw accelerometer, gyroscope, and magnetometer data, which the firmware fuses itself so the robot knows which way it is facing and how far it has turned. This information is used to keep the robot driving in a straight line and also give it a best guess as to where sunlight might be in accordance with magnetic north.

Distance to obstacles comes from four MaxBotix MB1020-000 ultrasonic sensors, mounted at the corners of the robot with two facing forward and two facing back. The MB1020-000 won out after a trade study because it fit the electrical and beam constraints without the roughly 100 dollar per unit price of most sensors that did. It runs on 3 to 5 V, draws about 2 mA, reaches out to 6.5 m, and has a 42 degree beam, all of which suit a slow moving, power conscious platform where update rate matters less than staying cheap and low draw. It also supports sensor chaining, so the four units trigger sequentially to keep from hearing each other's echoes without any complicated timing logic in firmware. That chaining still runs even

though the sensors also sit behind the 4:1 mux, since the mux handles which line the firmware reads while the chaining handles when each sensor fires.

Beam angle drove the selection, because it sets how much of the robot's surroundings the sensors actually cover. Four sensors leave three blind regions, one on the far left, one on the far right, and a triangle directly in front, and mounting the sensors with a small outward offset trades some of the front blind distance for tighter side coverage. For a pair of sensors with beam angle β and an outward offset, the angle the robot cannot see on each side is.

$$\alpha = (180 - \beta) / 2 - \text{offset} \quad (4)$$

and the interior angle of the front blind triangle grows with the same offset,

$$\delta = (180 - \beta) / 2 + \text{offset} \quad (5)$$

With the sensors spaced a distance d apart, the front blind triangle splits into two right triangles, so the deepest the front blind spot reaches is

$$\text{MaxBlindDist} = (d / 2) \cdot \tan(\delta) \quad (6)$$

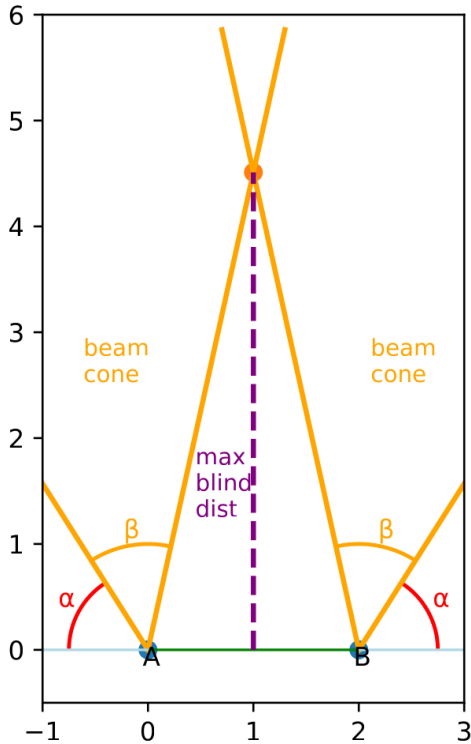


Fig. 3 Two Ultrasonic Sensor Beam Cone Coverage

A separate concern is the beam hitting the ground and reading it as an obstacle, which is fixed by tilting the sensors up so the bottom edge of the cone does not reach the ground until a chosen distance. With a mount height h and a target ground contact distance L , the

required vertical offset comes from the same right triangle geometry,

$$\text{offset} = \arctan(L / h) - (180 - \beta) / 2 \quad (7)$$

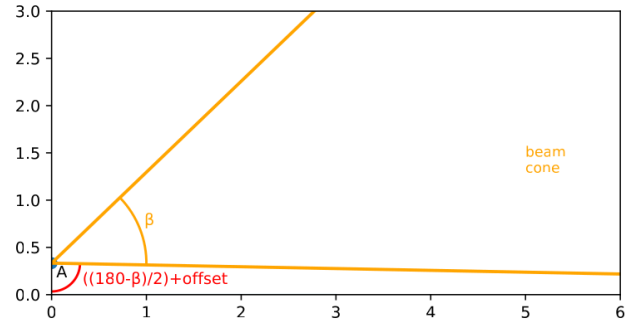


Fig. 4 Ultrasonic Sensor Angle Pointing Towards the Ground

Sweeping β and the horizontal offset through these equations, we set a design goal of keeping the front blind distance near five feet while holding α at or below 60 degrees. The mid range beam angles hit that best, so the final configuration pairs the MB1020-000's 42 degree beam with a 10 degree horizontal offset, and a 20 degree vertical offset, which balances coverage, precision, and false reading rejection for a robot moving this slowly.

The last channel is the robot watching itself. Battery and power telemetry come from a Texas Instruments INA228-Q1, a 20 bit power and energy monitor on the I2C bus, set up in a high side topology with a 15 mΩ shunt on the main line between the LiFePO4 battery and the load. The shunt was sized to use about 73 percent of the device's plus or minus 40.96 mV input range at the expected 2 A peak draw, which covers four drive motors at roughly 0.3 A each plus about 0.3 A of overhead, while leaving headroom for motor startup surges. Both I2C lines are pulled up to 5 V through 10 kΩ resistors, and a 100 nF cap sits at the supply pin to knock down switching noise from the motors. The part accumulates charge and energy in its own hardware registers, so it does real time coulomb counting for state of charge without loading the main processor, and it tracks energy leaving the battery during driving and coming back in during solar charging. This is the reading the whole power aware strategy depends on, and it is what feeds the net power in equation (1) and the move decision in equation (9).

C. Motor Drivers

Every moving part on the robot is a goBILDA Yellow Jacket brushed DC motor, and they all use the same driving scheme. Each motor is driven by an H-bridge driver board. The drivers were designed with a VN5019ATR driver chip which normally takes two directional inputs. Our custom design uses an onboard inverter so that one custom DIR input can generate a left or right direction without the need for two separate

signals. That means we only send a single direction line per motor, and the inverter produces the opposite line for us. The upside is that we cut the number of control lines the drivers need, which matters because pins are one of the tightest resources on the board once every sensor and peripheral takes its share. The tradeoff is that the driver's native stop state, where both direction inputs sit at the same level, is not reachable for us, since the inverter forces the two lines to always be opposites. We can still command forward or reverse, we just cannot use that built in stop.

The control signals do not run straight from the ESP32-S3 to the driver boards. An optocoupler sits between the main board and each driver board to keep the logic side electrically separated from the noisy motor side, which protects the microcontroller from the current spikes and back EMF the motors generate. The optocoupler also inverts every signal that crosses it, and that inversion changes how we stop a motor. Since we cannot use the driver's both lines stop state, we stop a motor by driving its PWM to zero instead. Because of the optocoupler inversion, getting a real zero at the driver means sending a high signal out of the ESP32-S3, since the optocoupler flips it to low on the other side. The four drive motors are the goBILDA 5204 series, rated 12 V and 18.14 Nm at 60 RPM through a 139:1 planetary gearbox. Paired with 4 inch wheels, that output speed sets the robot's top speed. Linear speed follows from the gearbox output RPM and the wheel diameter,

$$v = (N / 60) \cdot \pi \cdot D \quad (8)$$

where N is the output speed in RPM and D is the wheel diameter. At 60 RPM with a 4 inch (0.1016 m) wheel, this gives about 0.32 m/s, which lines up with our 0.3 m/s top speed target with a small margin. The panel pan and tilt motors are the goBILDA 5203 series, rated 12 V and 24.5 Nm at 30 RPM through a 188:1 planetary gearbox, since holding and swinging the panel against imbalance and wind needs more torque and less speed than rolling a wheel. Both motors ship with integrated quadrature encoders on 3.3 to 5 V logic. We only read one channel per encoder, channel A, and feed it into a PCNT pulse counter on the ESP32-S3 rather than reading both A and B. A normal quadrature setup needs both channels to figure out direction, but we already know which way each motor is turning because the firmware commanded it, so we set the PCNT unit's count direction to match the commanded direction and let it count the channel A pulses. Reading channel B would only tell us something we already know, so skipping it saves a pin per encoder and simplifies the wiring, and it has held up well in testing. The driver boards route direction and PWM through header pins, pull the high current battery connection through a terminal block, and land the motor wires on a screw terminal block.

Motor Direction Truth Table		
A	B	OUT
0	0	Undefined (Unused)
0	1	Forwards
1	0	Backwards
1	1	Undefined (Unused)

Fig. 5 Motor Driver Direction Truth Table

The motor is driven by a single GPIO pin which is fed through an inverter in order to save on pinouts. In this setup the motor driver sees two inputs but they are really just one inverted GPIO signal. Our truth table represents how we use a single DIR signal control scheme for slowing and stopping our motors rather than applying a current induced hard break with our undefined states.

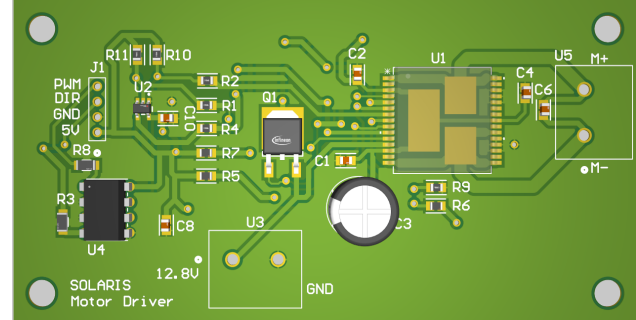


Fig. 6 Motor Driver 3D Layout

D. Mechanical Design

The chassis is a rigid cube built from aluminum T-slot extrusions. We picked T-slot for its strength to weight ratio and because its modularity lets us move or add components without machining new parts, which is exactly what an evolving prototype needs. The six faces are closed in with corrugated plastic panels. Four drive motors sit at the corners of the base for a stable four point drive and a low center of gravity under the panel. On top, the two motor pan and tilt assembly carries the 20 W panel, with one motor swinging the panel in azimuth and the other setting its elevation.

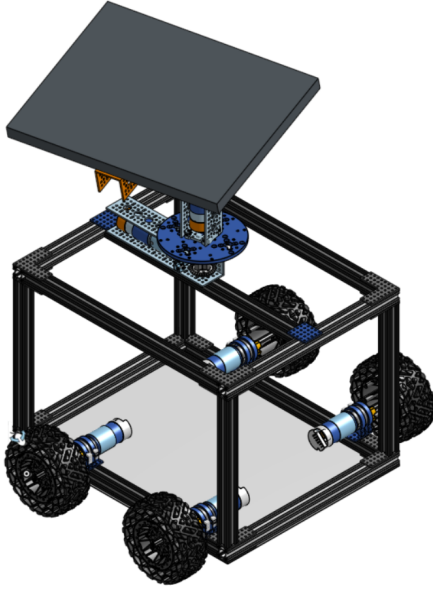


Fig. 5 SOLARIS Chassis Without Covers

IV. SOFTWARE ARCHITECTURE

The firmware is written entirely in C on top of ESP-IDF, built with PlatformIO for the ESP32-S3. C is the right call for the embedded side because it gives direct hardware access, predictable timing, and tight control over power, all of which the robot needs. The software is organized around FreeRTOS tasks and a mode state machine, because the robot's behavior really is a set of distinct modes with clear transitions between them. At boot, the firmware brings up the BLE stack, initializes the encoders, motor driver, and phototransistor subsystem, creates the shared mutex and the mode queue, and starts the sun tracking task and the driver task.

A. Mode State Machine

The robot runs in one of three operating modes, held in a mode state and switched by commands from the phone: Stationary, Manual, and Automatic. A driver task owns the state machine. It waits on a queue that the BLE mode characteristic feeds, so when the user picks a mode in the app, that choice lands in the queue and the driver task switches behavior. In Stationary, the robot stays put and runs sun tracking, pointing the panel at the strongest light without driving anywhere. In Manual, the user drives the robot directly, with throttle and steering coming down over BLE and sensor data going up to the app. In Automatic, the robot runs its own logic to decide when to relocate for better light. Manual can be selected at any time to take direct control, and the physical power switch cuts everything.

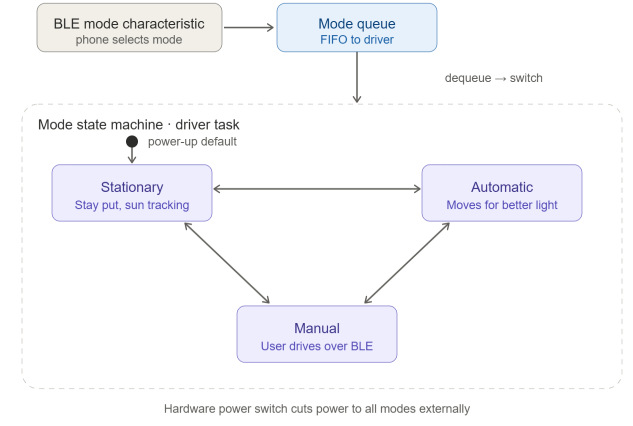


Fig. 6 BLE Controlled State Machine

B. Sun Tracking

Sun tracking runs in Stationary mode and is what keeps the panel aimed at the strongest available light. It is event driven rather than periodic: the tracking task blocks until the driver task notifies it, then reads the panel facing phototransistors and adjusts the gimbal. The panel carries four of the eight phototransistors, arranged as quadrants. The task compares the left pair against the right pair to decide whether to pan, and the top pair against the bottom pair to decide whether to tilt, and it only moves when the difference between a pair crosses a set tolerance, so small noise does not cause constant twitching. Before it moves the gimbal it takes the shared motor mutex, so the panel never swings while the wheels are driving.

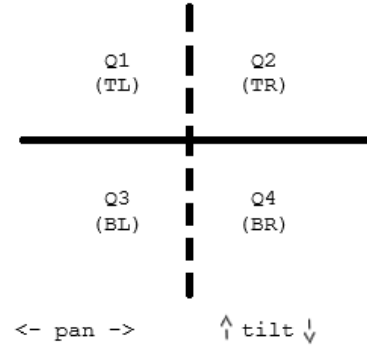


Fig. 6 Quadrant phototransistor map

Axis	Motor	Comparision	Action
Pan	Horizontal	$L > R$	Rotate left
$(Q1 + Q3) \text{ vs } (Q2 + Q4)$		$R > L$	Rotate right
		$ L - R < \text{Threshold}$	Hold
Tilt	Vertical	$T > B$	Tilt up

(Q1 + Q2) vs (Q3 + Q4)	B > T	Tilt down
	T - B < Threshold	Hold

Fig. 7 Pan and tilt comparison logic

C. Autonomous Relocation

Automatic mode is where the robot decides to move itself for better light, and it is the most involved piece of the design. The other four phototransistors, the ones not used for the panel, face outward from the chassis and are what the robot uses to decide which direction to head when it wants to relocate.

The decision to relocate is deliberately greedy. The robot is not running a full optimization or trying to predict the single best spot to sit. It watches the INA228 energy monitor and looks for a significant drop in power over a rolling window, comparing a short 30 second median against a longer 3 minute median so a brief flicker does not trigger a move but a real sustained drop does. When the drop crosses a threshold, the robot checks the current cloud coverage that the app pushed down from the weather backend before it acts,

$$\text{move if } (\bar{P}_{3min} - \bar{P}_{30s}) \geq P_{thresh} \text{ and } C \leq C_{max} \quad (9)$$

where C is the cloud coverage percentage and $\bar{P}$ is the windowed median power. A drop like that usually means the robot has fallen into shade or the sun has moved off the panel. The cloud coverage check is what makes the tradeoff work. If the sky is clear, moving is likely to help, so the robot commits. If cloud coverage is high, moving probably will not recover much power because the whole area is dim, so the robot may choose to stay put even though it is sitting in shade. That keeps the robot from burning energy chasing light that is not there.

When the robot does decide to move, it reads the four chassis phototransistors to pick the most promising direction, then rotates to face that way and drives. It does this in a sunk cost loop rather than a single attempt. After each step it rechecks the power against the earlier level. If power is recovered, the robot stops and stays. If it did not, it tries another step. It repeats this up to three times before giving up and settling where it is. This caps how much energy the robot is willing to gamble on a guess, which matters because the prediction is not guaranteed to be right every time.

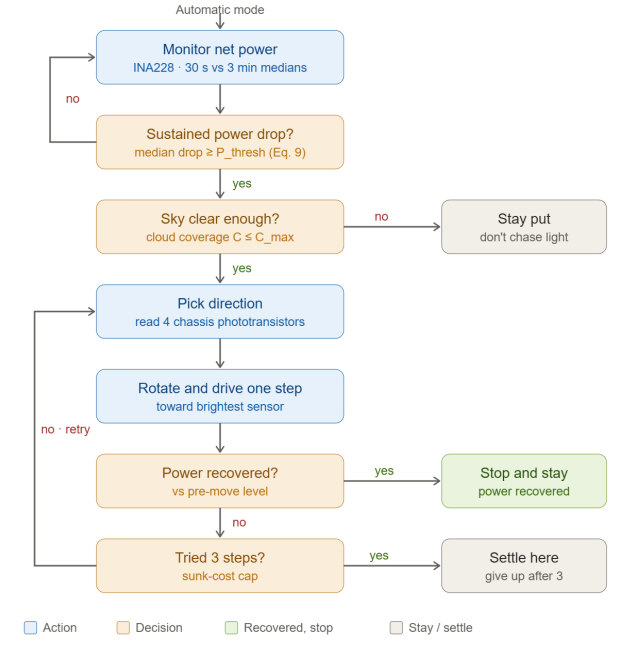


Fig. 8 Autonomous light seeking behavior diagram

D. Supporting Sensors and Algorithms

Several sensors back the driving logic. The INA228 power monitor is the input the relocation decision is built around, since the sustained power drop is what triggers a move in the first place. The ICM-20948 IMU provides heading, which the robot uses to rotate by a known amount when it picks a direction to drive. The four MB1020 ultrasonic sensors provide obstacle sensing so the robot can avoid collisions while relocating.

E. Telemetry Logs

Separate from anything the app requests, the robot keeps its own running log so it has a record even when no phone is connected. A dedicated logging task wakes once a minute and writes a snapshot of four values: battery level as a percentage, current net power draw from the power monitor, CPU temperature, and distance moved, where the distance comes from the encoders on the two front drive wheels so the log tracks how far the robot has actually traveled rather than how far it was told to go. Each snapshot is timestamped and appended to the log, which is written to flash so it survives power cycles and disconnects, which matters because the robot spends most of its time on its own with no phone in range. This log is the only source the app reads from. There is no separate live data path, so when the user opens or refreshes the app, what they see comes from the stored snapshots, which means the data is never more than about a minute stale since the log updates once a minute. When the BLE service comes up and the app requests the log, the stored snapshots are paged up to the phone through the telemetry protocol described in Section V, and once the

phone acknowledges it has everything, the robot clears the log to make room for the next stretch of operation.

V. WIRELESS COMMUNICATION

The link between the robot and the user is Bluetooth Low Energy, which fits because it is low power and the range we need is only a few meters. The firmware builds on the ESP-IDF NimBLE peripheral example, which gave us a working GAP and GATT skeleton with advertising, connection handling, and service registration [3]. We moved it into its own components so the SOLARIS firmware could add services without touching the init code, and named the device SOLARIS so it is obvious when scanning.

On top of that we wrote our own GATT service with six characteristics for the things the robot and phone need to exchange: telemetry notify, control, mode, time sync, weather, and manual control. Most of them are the phone writing down to the robot. Mode lets the app put SOLARIS into manual, stationary, or automatic. Manual control carries the joystick throttle and steering, sent one command at a time so the robot only reacts to the newest input. Time sync is a single Unix time packet whose only job is to set the ESP32's clock. Weather is forwarded from the app's backend API and handed off to the device: sunrise, sunset, cloud coverage as a percentage, and the chance of precipitation across the day, all based on the phone's latitude and longitude. SOLARIS uses that weather to decide whether it should actually be driving in automatic mode. Going the other way, the telemetry notify characteristic is how the robot pushes back the data worth watching: CPU temperature, battery level as a distance traveled,, and net power to show whether the robot is gaining or losing energy. The control characteristic drives the log transfer, which gets its own paragraph below.

Telemetry upload is the one piece that needed real protocol design, so we built it to behave like a small reliable transfer on top of BLE. The phone asks for a page of log records, the ESP32-S3 sends it, the phone acknowledges it, and only then does the ESP32-S3 send the next page. If the acknowledgment does not come back in time, the ESP32-S3 resends the current page. When the phone has everything, it sends a clear command and the robot wipes its log. This paged and acknowledged approach keeps records from being dropped or duplicated over a connection that can be flaky.

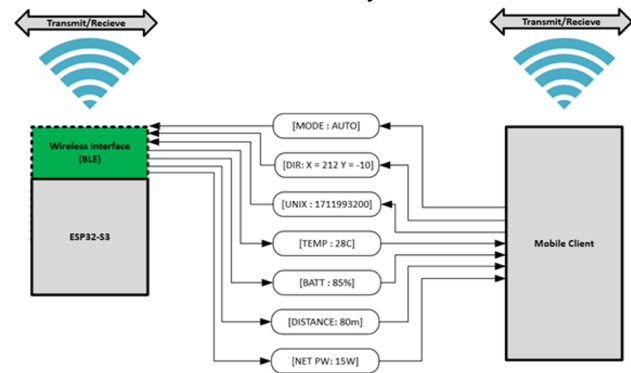


Fig. 6 SOLARIS telemetry flow diagram

VI. MOBILE APPLICATION AND BACKEND

The companion app is built in React Native and JavaScript so a single codebase runs on both iOS and Android. It talks to the robot directly over BLE and talks to a backend over HTTP. Because the BLE library is a custom native module that Expo Go cannot run, we use an EAS development build on a physical iPhone to get the native permissions, and we tested against the real ESP32-S3 throughout.

Authentication is handled by Clerk, which also scopes telemetry to each user so a person only sees their own robot data. On launch the app checks for a cached session and, if it finds one, goes straight to the dashboard, otherwise it offers sign in or sign up with email and password or an OAuth login. Once signed in, the app scans for a device named SOLARIS, connects, syncs the time, writes weather down to the robot, and pulls the telemetry log. The interface has a telemetry tab that shows battery, distance, net power, and CPU temperature as cards and charts, all sourced from the logged snapshots, and a mode tab that lets the user switch between stationary, automatic, and manual, with joysticks appearing in manual to stream throttle and steering down in real time.

The backend is a FastAPI service in Python, running as a local development build on our own machine rather than a hosted server for now. It links to a PostgreSQL database on Supabase, with credentials kept in environment variables and schema changes handled through migrations. It exposes the telemetry endpoints the app uses to store and retrieve a user's logged snapshots, and a weather endpoint that pulls forecast data from Open-Meteo based on the phone's GPS location, returning the cloud coverage, precipitation probability, and sunrise and sunset times that get written down to the robot. That cloud coverage number is what feeds the move decision in equation (9).

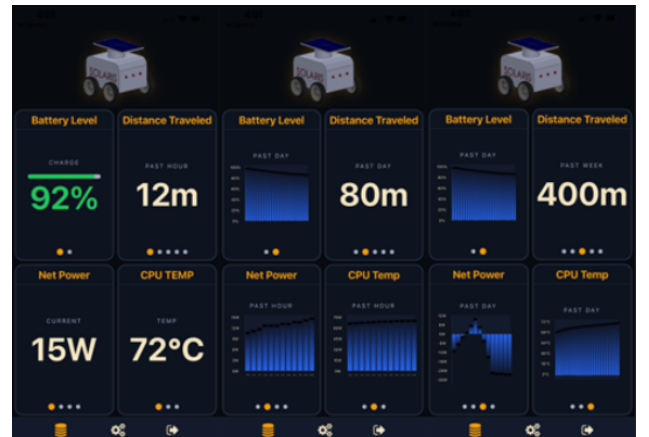


Fig. 7 SOLARIS Telemetry Screen

VII. STANDARDS AND DESIGN CONSTRAINTS

The design follows a handful of standards that shaped real decisions. The custom circuit boards were laid

out against IPC-2221 and its family of documents, which set generic PCB design rules for things like test point placement, clearances, and conductive plating, and which sort products into reliability classes that dictate how much inspection and margin a board needs [4]. Our boards target the general and dedicated service classes rather than the high reliability class, which is the right call for a student prototype. The software diagrams follow ISO flowchart symbol conventions so that anyone reading them, including our reviewers, sees the same meaning in each shape.

The hard constraints came from power, cost, and signal integrity. The power constraint is the whole thesis of the project, that the system has to stay energy positive over a day per equation (3), which is why the MPPT controller and the net power monitoring both exist. Signal integrity forced the split between the noisy motor bus and the clean sensor and logic rails, and it is the reason the drive signals cross an optocoupler before reaching the driver boards, since motor current spikes and back EMF will happily corrupt I2C traffic or reach back into the microcontroller if nothing isolates them. It also shaped the ADC layout, with the phototransistors and the ultrasonics assigned to separate ADC units so the two analog subsystems do not contend for the same peripheral. Cost was a constant pressure because the roughly 1000 dollar budget is self funded and split between four students. The two biggest line items were the chassis and the drive motors. We could not find a prebuilt frame that met our rigidity and weight targets for a price we could stomach, so we designed one from scratch out of aluminum, trading money for design time. The goBILDA drive motors were similarly hard to justify on cost, but their documentation, integrated encoders, and torque margin made them worth it over cheaper motors with weaker support.

VIII. RESULTS

Because SOLARIS is still in progress, the results so far are subsystem level verification rather than a single end to end field run. Several major subsystems already work together on the real hardware.

The wireless and control path is the furthest along. The full BLE stack works end to end: the robot advertises as SOLARIS, a phone connects, and the whole GATT protocol runs, including telemetry paging, mode set, time sync, weather write, and manual drive. We validated it in two stages. First we used nRF Connect on a phone to scan for the device, walk the full GATT table, and confirm every characteristic showed up with the right UUIDs and properties, then subscribed to notifications and wrote raw bytes to the mode, manual, and telemetry handlers to confirm the firmware parsed payloads correctly before the app was even ready. Then, with the app up, we ran the full connection flow with the phone console and the ESP serial monitor side by side and confirmed each step completed on both ends. Manual tank style driving over BLE works, with throttle and steering mixing into the four wheels in real time.

The panel sun tracking works in Stationary mode. The four quadrant phototransistors drive the pan and tilt gimbal through the MCPWM motor driver, and the panel moves to face the brightest light while respecting the tolerance band that keeps it from twitching on noise. The motor driver and the PCNT encoders were verified together on the bench by commanding known moves and confirming the counts tracked wheel and gimbal rotation cleanly, with the soft limits stopping the gimbal at its mechanical range. The in RAM telemetry log with NVS load was verified by paging records over BLE and checking them against the dashboard, confirming every card and chart showed the right numbers.

On the backend, the POST path was checked by uploading telemetry and confirming the rows landed in Supabase with correct values, and the weather endpoint was checked by calling it with a known location and confirming the response had the right shape, a full set of forecast entries, and valid sunrise and sunset times that matched Open-Meteo.

The remaining work is bringing the rest of the design online. The IMU, power monitor, and ultrasonic drivers are written and need to be integrated with the main firmware, the Automatic relocation logic needs to be built out against the power monitor and weather data, and the phototransistor load resistor value and the sensor thresholds that drive the move decision will be settled on the bench once the full robot is assembled. Taken together, the verified subsystems cover the communication, control, sensing, and data layers that the autonomous behavior rides on, which leaves the relocation logic as the main piece left to close.

IX. CONCLUSION

SOLARIS set out to test whether a mobile solar platform can earn back the energy it spends moving. The project is still in progress, but the design is far enough along to show how the parts fit and to prove out the backbone the rest depends on. The BLE link carries reliable paged telemetry and manual drive, the panel tracks the sun in Stationary mode through the phototransistor gimbal, the MCPWM motor driver and PCNT encoders run the six motors with soft limits, and the app and backend have been tested end to end. The relocation logic is designed around a greedy, power aware decision that leans on cloud coverage data and a capped retry loop to limit how much a bad guess can cost, and it is the main piece left to implement. The remaining work is wiring the sensor drivers into the firmware and closing that loop, then proving the energy positive claim of equation (3) in the field. Even at this stage, SOLARIS shows that the pieces of a self-sufficient, mobile solar robot fit together.

X. ACKNOWLEDGMENT

The authors thank our mentor Dr. Suboh Suboh for his guidance throughout the project, and our review committee, Dr. Maddox, Dr. Rakshan, and Dr. Sahawneh, for their feedback. We also thank the University of Central

Florida Department of Electrical and Computer Engineering for the resources that made this work possible.

REFERENCES

- [1] Jackery, "Solar Mars Bot," 2024. [Online]. Available: <https://www.jackery.com/pages/solar-mars-bot> (accessed Apr. 27, 2026).
- [2] I. Reda and A. Andreas, "Solar position algorithm for solar radiation applications," National Renewable Energy Laboratory, Golden, CO, USA, Tech. Rep. NREL/TP-560-34302, Jan. 2008. [Online]. Available: <https://docs.nrel.gov/docs/fy08osti/34302.pdf> (accessed Apr. 27, 2026).
- [3] Espressif Systems, "NimBLE-based host APIs — ESP-IDF programming guide," 2024. [Online]. Available: <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/bluetooth/nimble/index.html> (accessed Apr. 27, 2026).
- [4] Generic Standard on Printed Board Design, IPC-2221, 1998.
- [5] goBILDA, "5203 and 5204 series Yellow Jacket planetary gear motors," 2024. [Online]. Available: <https://www.gobilda.com/yellow-jacket-planetary-gear-motors/> (accessed Apr. 27, 2026).
- [6] Texas Instruments, "INA228 85-V, 20-bit, ultra-precise power/energy/charge monitor," datasheet SBOS736, 2021. [Online]. Available: <https://www.ti.com/lit/ds/symlink/ina228.pdf> (accessed Apr. 27, 2026).
- [7] Genasun, "GV-5 solar charge controller with MPPT," 2024. [Online]. Available: <https://sunforgellc.com/product/gv-5/> (accessed Apr. 27, 2026).
- [8] Open-Meteo, "Free weather API documentation," 2024. [Online]. Available: <https://open-meteo.com/en/docs> (accessed Apr. 27, 2026).
- [9] Clerk, "Authentication and user management documentation," 2024. [Online]. Available: <https://clerk.com/docs> (accessed Apr. 27, 2026).